

My Notes on Backpropagation

Daniel Zhang

2026

Abstract

These notes record my working understanding of backpropagation as a local, layer-by-layer process rather than a set of formulas to be solved symbolically. I start from the forward pass (the composition of linear and activation steps that produces a prediction), define the squared-error loss, and then trace the backward pass as the chain rule applied in reverse, alternating an activation (error-signal) stage and a linear (transport) stage to produce the parameter gradients dW and db at each layer. Matrix forms make the tensor shapes concrete, and the cached forward-pass values Z and A are reused to avoid recomputation. A simplified, fully-worked one-neuron example then carries a single training example through the forward pass, the backward pass, and one gradient-descent update to show the mechanics end to end.

Contents

1	Overview	2
2	Forward Pass: The Prediction or Inference Phase	3
3	Loss Function: Forming the Error Signal	4
4	The Backward Pass: Passing the Error Signal Backward For Learning	5
4.1	Summary of the Backpropagation Process	7
5	Updating the Parameters: The Training Step	7
6	Worked Example	7

Backpropagation as Local Differential State Machine

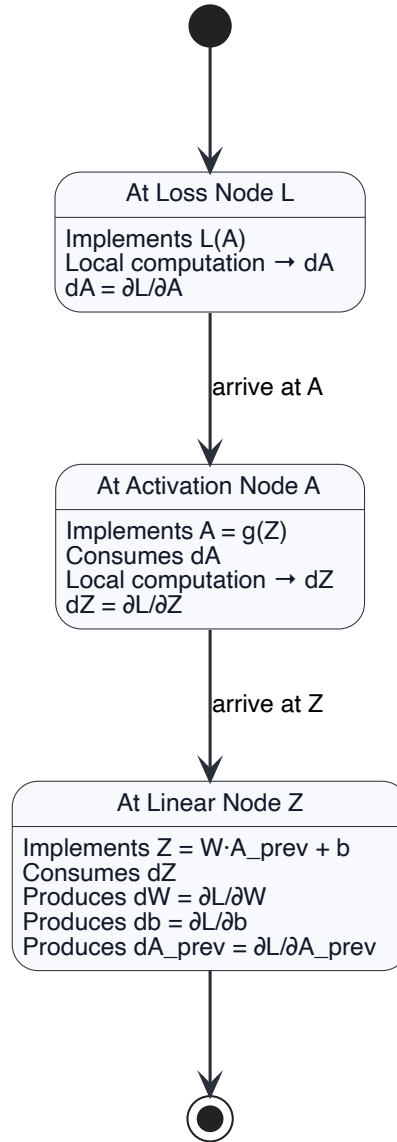


Figure 1: The backward pass as a state machine: each node implements a local forward operation, consumes the gradient arriving from its successor, and emits the gradient(s) feeding its predecessor. A roadmap for the layer-by-layer detail that follows.

1 Overview

Backpropagation resembles the scientific method. An experiment is run, error is measured, parameters are adjusted, and the process repeats.

Many disciplines follow this same loop:

evaluate error $\rightarrow$ adjust parameters $\rightarrow$ test again.

These notes summarize my understanding while studying backpropagation.

I find writing the equations by hand enables relating to the math differently than just reading it. Initially, I approached the subject through closed-form symbolic math, attempting to solve the equations symbolically.

Like working on an engine, there is a difference between the theoretical description and the greasy mechanics.

Progress came when I treated the equations as an outline of a process rather than formulas meant to be solved analytically.

Instead of reflexively thinking of the derivatives as formulas such as $\frac{d}{dx}(x^2) = 2x$, the limit form captures more of the numerical process that applies to machine learning. For the example of x^2 , the limit form is:

$$\lim_{\Delta x \rightarrow 0} \frac{(x + \Delta x)^2 - x^2}{\Delta x} = \lim_{\Delta x \rightarrow 0} (2x + \Delta x) = 2x$$

As the tiny step Δx gets closer and closer to zero, that fraction settles into the exact slope at that point.

You can plug in different values to convince yourself. Concrete examples help:

- If you are at $x = 3$ and take a tiny step of $\Delta x = 0.1$, the average slope is $\frac{(3.1)^2 - (3^2)}{0.1} = 6.1$.
- If you take an even smaller step, $\Delta x = 0.01$, the average slope is $\frac{(3.01)^2 - (3^2)}{0.01} = 6.01$.
- As the step shrinks toward zero, those average slopes settle at 6.00, which is the exact slope at $x = 3$ when computed with the closed-form formula shown above.

Our goal with backpropagation is simple: make predictions closer to targets by adjusting a model's parameters, including weights and biases.

Formally, this means transmitting information through a nonlinear space using linear operations of tiny changes, or perturbations, intending to minimize the loss function of the network.

These changes enable learning by comparing predictions to the target and using the error to update the weights and biases.

We start with data and its ground-truth target.

2 Forward Pass: The Prediction or Inference Phase

The forward pass is used to generate the predicted output and activation values of each layer. Here, Z is the weighted sum ($W^{[l]}A^{[l-1]} + b^{[l]}$) and A is the activated output ($g^{[l]}(Z^{[l]})$). The computation graph of the forward pass is:

$$W, b \rightarrow Z \rightarrow A \rightarrow \mathcal{L} \quad (1)$$

where $\mathcal{L}$ is the loss, not to be confused with the total number of layers L in the network.

It is a composition of functions with many nested parentheses. It's mostly bookkeeping: the same local pattern repeats at each layer, but composition makes the nesting matter.

$$\hat{Y} = f^{[L]} \left(f^{[L-1]} \left(\dots f^{[2]} \left(f^{[1]}(X) \right) \dots \right) \right) \quad (2)$$

Expanding f as $g(Z^{[l]}) = g(W^{[l]}A^{[l-1]} + b^{[l]})$ gives:

$$\hat{Y} = g^{[L]} \left(W^{[L]} g^{[L-1]} \left(W^{[L-1]} \dots g^{[1]} \left(W^{[1]}X + b^{[1]} \right) \dots + b^{[L-1]} \right) + b^{[L]} \right) \quad (3)$$

where $g^{[l]}$ is the gate function of layer l .

Forward propagation goes from the innermost function to the outermost function. The forward pass for each layer is computed using the following equations with m being the number of training examples:

$$A^{[0]} = X, \quad X \in \mathbb{R}^{n_x \times m} \quad (4)$$

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \quad (5)$$

$$A^{[l]} = g^{[l]}(Z^{[l]}) \quad (6)$$

$$Z^{[l]} \in \mathbb{R}^{n_l \times m}, \quad W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}, \quad A^{[l-1]} \in \mathbb{R}^{n_{l-1} \times m}, \quad b^{[l]} \in \mathbb{R}^{n_l \times 1}, \quad A^{[l]} \in \mathbb{R}^{n_l \times m} \quad (7)$$

$$A^{[L]} \in \mathbb{R}^{n_y \times m}, \quad n_y \text{ is the number of units in the output layer} \quad (8)$$

Concrete form (the bias is added to each column of the activation tensor with examples arranged as columns):

$$Z^{[l]} = \begin{bmatrix} z_1^{[l](1)} & \cdots & z_1^{[l](m)} \\ z_2^{[l](1)} & \cdots & z_2^{[l](m)} \\ \vdots & \ddots & \vdots \\ z_{n_l}^{[l](1)} & \cdots & z_{n_l}^{[l](m)} \end{bmatrix} = \begin{bmatrix} w_{11}^{[l]} & \cdots & w_{1n_{l-1}}^{[l]} \\ w_{21}^{[l]} & \cdots & w_{2n_{l-1}}^{[l]} \\ \vdots & \ddots & \vdots \\ w_{n_l 1}^{[l]} & \cdots & w_{n_l n_{l-1}}^{[l]} \end{bmatrix} \begin{bmatrix} a_1^{[l-1](1)} & \cdots & a_1^{[l-1](m)} \\ a_2^{[l-1](1)} & \cdots & a_2^{[l-1](m)} \\ \vdots & \ddots & \vdots \\ a_{n_{l-1}}^{[l-1](1)} & \cdots & a_{n_{l-1}}^{[l-1](m)} \end{bmatrix} + \begin{bmatrix} b_1^{[l]} & \cdots & b_1^{[l]} \\ b_2^{[l]} & \cdots & b_2^{[l]} \\ \vdots & \ddots & \vdots \\ b_{n_l}^{[l]} & \cdots & b_{n_l}^{[l]} \end{bmatrix} \quad (9)$$

Applying the activation function element-wise gives the activation matrix:

$$A^{[l]} = g(Z^{[l]}) = \begin{bmatrix} g(z_1^{[l](1)}) & g(z_1^{[l](2)}) & \cdots & g(z_1^{[l](m)}) \\ g(z_2^{[l](1)}) & g(z_2^{[l](2)}) & \cdots & g(z_2^{[l](m)}) \\ \vdots & \vdots & \ddots & \vdots \\ g(z_{n_l}^{[l](1)}) & g(z_{n_l}^{[l](2)}) & \cdots & g(z_{n_l}^{[l](m)}) \end{bmatrix} \quad (10)$$

The training example is shown as the superscript (i) , the layer as $[l]$, and the neuron in the layer as the subscript.

The final activation $A^{[L]}$ is the network prediction, also written as $\hat{Y}$.

3 Loss Function: Forming the Error Signal

The loss function is given by taking the difference of the predicted output generated by the forward pass and the target output in the training set.

$$\mathcal{L}(\hat{Y}, Y) = \frac{1}{2}(\hat{Y} - Y)^2 \quad (11)$$

For a single training example, the loss function is:

$$\mathcal{L}^{(i)}(\hat{Y}^{(i)}, Y^{(i)}) = \frac{1}{2}(\hat{Y}^{(i)} - Y^{(i)})^2 \quad (12)$$

where Y is the source of truth and $\hat{Y}$ is the predicted output.

The extra 2 on the bottom cancels the square in the numerator when taking the derivative. Squaring makes the scalar loss smooth and convex as a function of the prediction, and the derivative is linear in the error.

For this scalar prediction case, convex means the bowl-shaped loss function has a single global minimum with respect to $\hat{Y}$ for a fixed target Y .

That is useful because the local slope always points toward the target: if $\hat{Y}$ is too large, the gradient is positive; if $\hat{Y}$ is too small, the gradient is negative.

This convexity is a local property. It does not mean the full neural-network training problem is convex in the parameters W, b ; it only means this particular loss is well-behaved as a function of the prediction.

Loss $\mathcal{L}$ is defined per example; cost $\mathcal{J}$ is the average loss across the full training set. The cost function averages those per-example losses across the training set:

$$\mathcal{J}(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots, W^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}^{(i)}(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots, W^{[L]}, b^{[L]}) \quad (13)$$

Each $\mathcal{L}^{(i)}$ uses the i -th training example (its input and target), and m is the number of training examples.

With this cost in hand, training follows a loop: run a forward pass to get predictions, compute loss, backpropagate gradients, update W, b , and repeat over many batches. Each loop is one step of learning, nudging parameters in the direction that lowers the total cost $\mathcal{J}$.

4 The Backward Pass: Passing the Error Signal Backward For Learning

No learning takes place without sending an error signal backward through the network so that the parameters can be updated.

Backpropagation uses the fixed computation graph from the forward pass whose operations are locally differentiable. The structure matches the forward pass during the backward phase. Each gradient expression is defined with respect to a local operation in the graph.

The values $Z^{[l]}$ and $A^{[l]}$ are fixed and known from the forward pass. This allows gradient computations within a layer to be parallelized, while dependencies between layers enforce a sequential reverse order.

The computation graph of the backpropagation process is:

$$\mathcal{L} \rightarrow A \rightarrow Z \rightarrow W, b \quad (14)$$

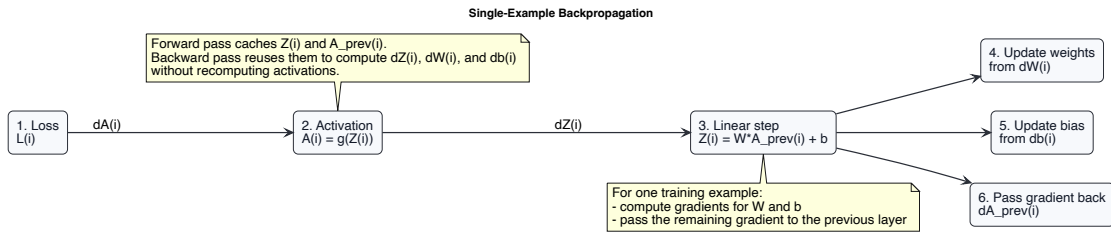


Figure 2: Single-example view of the backward computation graph $\mathcal{L} \rightarrow A \rightarrow Z \rightarrow W, b$. The forward-pass values $Z^{[l]}$ and $A^{[l-1]}$ are cached and reused, so the backward pass computes dZ , dW , and db without recomputing activations.

Backpropagation is the chain rule applied in reverse through the composed functions of the network.

The backpropagation process alternates between the signal error mode, mapped by $\frac{\partial \mathcal{L}}{\partial A^{[l]}}$ and the transport stage, mapped by $\frac{\partial \mathcal{L}}{\partial Z^{[l]}}$ to propagate the error signal backwards through the network until the input layer is reached.

In differential form, a tiny change in the loss is

$$d\mathcal{L} = \frac{\partial \mathcal{L}}{\partial A^{[L]}} dA^{[L]}. \quad (15)$$

Here the gradient $\frac{\partial \mathcal{L}}{\partial A^{[L]}}$ is the coefficient that scales a small change in $A^{[L]}$.

Because we differentiate with respect to activations, the gradient has the same shape as the activation. For the input layer and a general layer:

$$A^{[0]} = X \in \mathbb{R}^{n_x \times m}, \quad \frac{\partial \mathcal{L}}{\partial A^{[0]}} \in \mathbb{R}^{n_x \times m} \quad (16)$$

$$A^{[l]} \in \mathbb{R}^{n_l \times m}, \quad \frac{\partial \mathcal{L}}{\partial A^{[l]}} \in \mathbb{R}^{n_l \times m} \quad (17)$$

where n_x is the number of features and n_l is the number of units in the layer.

We now track how error moves backward layer by layer. For $l = L, L-1, \dots, 1$:

$$\boxed{\frac{\partial \mathcal{L}}{\partial A^{[l]}} \rightarrow \frac{\partial \mathcal{L}}{\partial Z^{[l]}} \Rightarrow \left(\frac{\partial \mathcal{L}}{\partial W^{[l]}}, \frac{\partial \mathcal{L}}{\partial b^{[l]}} \right) \rightarrow \frac{\partial \mathcal{L}}{\partial A^{[l-1]}}} \quad (18)$$

First, the output gradient is taken with respect to the activation, so it has the same shape as $A^{[L]}$. From there, the error signal stage is the dA step, which tells us how the loss changes with activations. The transport stage is the dZ step, which moves that signal through the layer via the derivative of the gate. By alternating dA and dZ , we carry the error backward and compute parameter gradients along the way.

$$\frac{\partial \mathcal{L}}{\partial A^{[L]}} = \frac{\partial \mathcal{L}(A^{[L]}, Y)}{\partial A^{[L]}} \quad (19)$$

$$dZ^{[l]} = \frac{\partial \mathcal{L}}{\partial A^{[l]}} \odot g'(Z^{[l]}) \quad (20)$$

$$\left(\frac{\partial \mathcal{L}}{\partial W^{[l]}}, \frac{\partial \mathcal{L}}{\partial b^{[l]}} \right) = \left(\frac{1}{m} dZ^{[l]} A^{[l-1]T}, \frac{1}{m} \sum_{i=1}^m dZ^{[l]} \right) \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial A^{[l-1]}} = W^{[l]T} dZ^{[l]} \quad (22)$$

Concrete form for the backward signal in a layer:

$$dZ^{[l]} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial A_1^{[l](1)}} g'(z_1^{[l](1)}) & \frac{\partial \mathcal{L}}{\partial A_1^{[l](2)}} g'(z_1^{[l](2)}) & \dots & \frac{\partial \mathcal{L}}{\partial A_1^{[l](m)}} g'(z_1^{[l](m)}) \\ \frac{\partial \mathcal{L}}{\partial A_2^{[l](1)}} g'(z_2^{[l](1)}) & \frac{\partial \mathcal{L}}{\partial A_2^{[l](2)}} g'(z_2^{[l](2)}) & \dots & \frac{\partial \mathcal{L}}{\partial A_2^{[l](m)}} g'(z_2^{[l](m)}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{L}}{\partial A_{n_l}^{[l](1)}} g'(z_{n_l}^{[l](1)}) & \frac{\partial \mathcal{L}}{\partial A_{n_l}^{[l](2)}} g'(z_{n_l}^{[l](2)}) & \dots & \frac{\partial \mathcal{L}}{\partial A_{n_l}^{[l](m)}} g'(z_{n_l}^{[l](m)}) \end{bmatrix} \quad (23)$$

Concrete form for transporting the signal back to the previous activation:

$$\frac{\partial \mathcal{L}}{\partial A^{[l-1]}} = W^{[l]T} dZ^{[l]} = \begin{bmatrix} w_{11}^{[l]} & w_{21}^{[l]} & \dots & w_{n_l 1}^{[l]} \\ w_{12}^{[l]} & w_{22}^{[l]} & \dots & w_{n_l 2}^{[l]} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1 n_{l-1}}^{[l]} & w_{2 n_{l-1}}^{[l]} & \dots & w_{n_l n_{l-1}}^{[l]} \end{bmatrix} \begin{bmatrix} dZ_1^{[l](1)} & dZ_1^{[l](2)} & \dots & dZ_1^{[l](m)} \\ dZ_2^{[l](1)} & dZ_2^{[l](2)} & \dots & dZ_2^{[l](m)} \\ \vdots & \vdots & \ddots & \vdots \\ dZ_{n_l}^{[l](1)} & dZ_{n_l}^{[l](2)} & \dots & dZ_{n_l}^{[l](m)} \end{bmatrix} \quad (24)$$

4.1 Summary of the Backpropagation Process

In summary, the backpropagation process is:

$$\boxed{\frac{\partial \mathcal{L}}{\partial A^{[l-1]}} = W^{[l]T} \left(\frac{\partial \mathcal{L}}{\partial A^{[l]}} \odot g'(Z^{[l]}) \right)} \quad (25)$$

During backpropagation, passing through the gate function is the error-signal stage and multiplying by the weights is the transport stage. It is exactly the reverse of the forward pass.

The error-signal stage produces $dZ^{[l]}$, and the transport stage produces $dA^{[l-1]}$.

The values of $Z^{[l]}$ and $A^{[l]}$ from the forward pass are cached because the backward pass needs them to evaluate $g'(Z^{[l]})$ and the parameter gradients. Without those saved intermediate values, the backward pass would have to recompute the forward activations.

Put simply: the gate applies the activation slope, and the weights map the signal backward to the previous layer.

Backpropagation alternates $dA \rightarrow dZ \rightarrow dA$ as it moves backward through layers, starting from $dA^{[L]}$. Here the inner term $\frac{\partial \mathcal{L}}{\partial A^{[l]}} \odot g'(Z^{[l]})$ is $dZ^{[l]}$. The recurring process is:

$$dA^{[l]} \rightarrow dZ^{[l]} \rightarrow (dW^{[l]}, db^{[l]}, dA^{[l-1]}) \rightarrow dZ^{[l-1]} \quad (26)$$

For cleaner notation, the adjoint variables (e.g. dZ , dW , dA) are used as shorthand for partial derivatives of the loss, with the explicit partial-derivative notation suppressed once the meaning is clear.

5 Updating the Parameters: The Training Step

Parameters are updated using the gradients computed in the backward pass. For consistency with the cost function defined above, it is cleaner to write the update in terms of $\mathcal{J}$, the batch-average cost:

$$W^{[l]} \leftarrow W^{[l]} - \alpha \frac{\partial \mathcal{J}}{\partial W^{[l]}} \quad (27)$$

$$b^{[l]} \leftarrow b^{[l]} - \alpha \frac{\partial \mathcal{J}}{\partial b^{[l]}} \quad (28)$$

If training uses a mini-batch, then $\mathcal{J}$ here means the average loss over that mini-batch. If training uses the full dataset in one step, then it means the full-dataset cost.

6 Worked Example

Setting aside the underlying equations, backpropagation consists of many simple numerical operations that can be performed on a calculator. Here is a one-neuron example using one training example to show the full forward and backward pass numerically.

Let

$$x = 2, \quad w = 3, \quad b = 1, \quad y = 4.$$

Use the identity activation $g(z) = z$, so $g'(z) = 1$. Then the forward pass is

$$z = wx + b = 3 \cdot 2 + 1 = 7, \quad a = g(z) = 7.$$

The squared-error loss is

$$\mathcal{L} = \frac{1}{2}(a - y)^2 = \frac{1}{2}(7 - 4)^2 = \frac{9}{2} = 4.5.$$

Now backpropagate. First compute the output gradient:

$$\frac{\partial \mathcal{L}}{\partial a} = a - y = 7 - 4 = 3.$$

Since the activation is the identity, the pre-activation gradient is

$$dz = \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} g'(z) = 3 \cdot 1 = 3.$$

Then the parameter gradients are

$$dw = \frac{\partial \mathcal{L}}{\partial w} = dz \cdot x = 3 \cdot 2 = 6,$$

$$db = \frac{\partial \mathcal{L}}{\partial b} = dz = 3.$$

The gradient passed to the input is

$$dx = \frac{\partial \mathcal{L}}{\partial x} = w \cdot dz = 3 \cdot 3 = 9.$$

If the learning rate is $\alpha = 0.1$, the updated parameters are

$$w \leftarrow 3 - 0.1(6) = 2.4, \quad b \leftarrow 1 - 0.1(3) = 0.7.$$

Running the forward pass again gives

$$z_{\text{new}} = 2.4 \cdot 2 + 0.7 = 5.5, \quad a_{\text{new}} = 5.5,$$

and the new loss is

$$\mathcal{L}_{\text{new}} = \frac{1}{2}(5.5 - 4)^2 = \frac{1}{2}(1.5)^2 = 1.125.$$

So one gradient step moved the prediction from 7 toward the target 4 and reduced the loss from 4.5 to 1.125.

License

Copyright (c) 2026 Daniel Zhang. Creative Commons Attribution 4.0 International License (CC BY 4.0): <https://creativecommons.org/licenses/by/4.0/>. Source: <https://github.com/d108>